GIS Data Science Portfolio Project

PROJECT 01

Urban Flood Risk & Vulnerability Mapping

A complete step-by-step beginner's guide to building a spatial machine learning pipeline that predicts flood risk using open government data, GeoPandas, Scikit-learn, and interactive web maps.

⏱ Total Time	Language	Difficulty	Data
~6–10 hours	Python 3.9+	Beginner–Intermediate	100% Free & Open

Introduction

Flooding is one of the costliest natural disasters in the United States, affecting millions of people every year. Traditional flood maps from FEMA are static — they don't capture the full picture of who is most vulnerable. In this project, you will build a dynamic, data-driven flood vulnerability model that combines spatial features (proximity to water, elevation) with socioeconomic data to produce a composite risk score for every census tract in Harris County, Texas (the Houston metro area — one of the most flood-prone regions in the US).

By the end of this project, you will have:

- Downloaded and processed real government GIS data (FEMA flood zones, elevation, census demographics)
- Engineered meaningful spatial features using buffer analysis and spatial joins
- Applied spatial autocorrelation analysis (Moran's I) to understand geographic clustering of risk
- Trained a Random Forest machine learning model to predict flood vulnerability scores
- Built an interactive choropleth web map that any stakeholder can explore

> **Why Harris County, Texas?**
> Houston experienced three catastrophic 500-year floods in three years (2015, 2016, 2017 — including Hurricane Harvey). This makes it the perfect case study: the data is rich, the stakes are real, and the analysis is directly relevant to urban planners, emergency managers, and insurers.

What You Will Build

At the end of this tutorial, you will produce three key outputs that go directly into your portfolio:

#	Output	Description
1	**Annotated Jupyter Notebook**	Full reproducible pipeline from raw data download to final model — this IS your portfolio piece.
2	**LISA Hotspot Map**	A Local Moran's I cluster map showing statistically significant flood-risk hotspots and cold spots across Harris County.
3	**Interactive Folium Map**	A shareable HTML web map with color-coded vulnerability scores per census tract. Click a tract to see its score breakdown.

Prerequisites

You do NOT need to be a GIS expert to follow this guide. You do need:

- Basic Python: you should be comfortable with variables, functions, loops, and importing libraries.
- Familiarity with Pandas DataFrames (reading CSVs, filtering rows, groupby).
- A free Google account (we'll work in Google Colab — no local installation required) OR Python 3.9+ installed locally.

First time with GIS? That's fine.

Every new concept in this guide is explained in a 'Concept' box before you use it in code. By the end, you'll understand what a GeoDataFrame, a spatial join, a CRS, a buffer, and Moran's I actually are.

STEP 0

Environment Setup

Install libraries and create your project notebook

⏱ Estimated time: 15–20 minutes

Option A — Google Colab (Recommended for Beginners)

Google Colab runs entirely in your browser — no installation needed. Go to colab.research.google.com, click 'New Notebook', and paste each code block as a new cell.

> 💡 **Saving your work in Colab**
> Go to File → Save a copy in Drive. This saves your notebook to your personal Google Drive so you don't lose progress.

Option B — Local Jupyter Notebook

If you prefer to work locally, open your terminal and run:

```
# Create a new folder for the project
mkdir flood_risk_project
cd flood_risk_project

# Create a virtual environment (keeps your project dependencies tidy)
python -m venv venv
source venv/bin/activate    # On Windows: venv\Scripts\activate

# Launch Jupyter
jupyter notebook
```

Installing Required Libraries

In a Colab cell (or terminal if local), run the following installation command. This only needs to be run once:

```
# ── Install all required libraries ──────────────────────────────────────
!pip install geopandas folium esda libpysal scikit-learn matplotlib
!pip install contextily osmnx censusdatadownloader shapely pysal

# ── Verify installations ────────────────────────────────────────────────
import geopandas as gpd
import pandas as pd
import numpy as np
import folium
import esda
import libpysal
from sklearn.ensemble import RandomForestRegressor
import matplotlib.pyplot as plt

print('✅ All libraries loaded successfully!')
```

```
print(f'   GeoPandas version: {gpd.__version__}')
```

✅ Expected Output: Library Check

```
✅ All libraries loaded successfully!
   GeoPandas version: 0.14.0
```

⚠ If you get an import error

Run the !pip install command again for the specific library that failed. If you see a warning about 'libspatialindex', re-run: !pip install Rtree

STEP 1

Download & Load the Data

Acquire FEMA flood zones, census tracts, and water body data

⏱ Estimated time: 30–45 minutes

Concept: What is a Shapefile?

A shapefile is the most common format for storing geographic vector data (points, lines, polygons). It actually consists of multiple files (.shp, .dbf, .prj, etc.) that must be kept together. GeoPandas can read shapefiles directly with one line of code.

1.1 — Set Up Imports and Configuration

Start your notebook with this standard imports block. Always do this in the first cell:

```
# ── Standard imports ──────────────────────────────────────────────────────
import geopandas as gpd
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import folium
from folium.plugins import FloatImage
import requests, zipfile, io, os
import warnings
warnings.filterwarnings('ignore')

# ── Project configuration ─────────────────────────────────────────────────
# Harris County FIPS code (Texas = 48, Harris County = 201)
STATE_FIPS  = '48'
COUNTY_FIPS = '201'
COUNTY_NAME = 'Harris County, TX'

# We'll work in this Coordinate Reference System (CRS)
# EPSG:32615 = UTM Zone 15N — best for Houston area (units: meters)
PROJ_CRS = 'EPSG:32615'

print(f'Project: Flood Vulnerability — {COUNTY_NAME}')
```

Concept: What is a CRS (Coordinate Reference System)?

A CRS defines how geographic coordinates map to the real Earth. EPSG:4326 uses latitude/longitude in degrees (good for displaying on web maps). EPSG:32615 (UTM Zone 15N) uses meters — much better when you need to measure distances or create buffers, because you can say 'give me everything within 1,000 meters' directly. We'll reproject between these two throughout the project.

1.2 — Download FEMA Flood Zone Data

FEMA's National Flood Hazard Layer (NFHL) is the authoritative source for official US flood zones. We'll download it directly via FEMA's API:

```
def download_fema_flood_zones(state_fips, county_fips):
    """
    Downloads FEMA NFHL flood zone polygons for a specific county.
    Returns a GeoDataFrame with flood zone designations.
    """
    print('📥 Downloading FEMA flood zone data...')

    # FEMA NFHL REST API endpoint for Flood Zone polygons (Layer 28)
    base_url = ('https://hazards.fema.gov/gis/nfhl/rest/services/'
                'public/NFHL/MapServer/28/query')

    # Build query: filter to Harris County by FIPS code
    params = {
        'where': f"DFIRM_ID LIKE '{state_fips}{county_fips}%'",
        'outFields': 'FLD_ZONE,ZONE_SUBTY,STUDY_TYP',
        'f': 'geojson',
        'returnGeometry': 'true',
        'resultRecordCount': 4000,
        'outSR': '4326',
    }

    response = requests.get(base_url, params=params, timeout=120)
    gdf = gpd.read_file(io.BytesIO(response.content))
    gdf = gdf.to_crs(PROJ_CRS)  # Reproject to meters

    print(f'   ✅ Downloaded {len(gdf):,} flood zone polygons')
    print(f'   Unique flood zone types: {gdf["FLD_ZONE"].unique()}')
    return gdf

flood_zones = download_fema_flood_zones(STATE_FIPS, COUNTY_FIPS)
flood_zones.head(3)
```

✅ Expected Output: FEMA API Response

```
📥 Downloading FEMA flood zone data...
   ✅ Downloaded 12,847 flood zone polygons
   Unique flood zone types: ['AE' 'X' 'AO' 'VE' 'A' 'D']
```

📚 Concept: FEMA Flood Zone Types

Zone AE = High-risk (1% annual chance flood, with Base Flood Elevation). Zone AO = High-risk shallowly flooded areas. Zone X (shaded) = Moderate risk (0.2% annual chance). Zone X (unshaded) = Minimal risk. Zone D = Undetermined. For this analysis, Zone AE and AO represent the most dangerous areas.

1.3 — Download Census Tract Boundaries with Demographics

We'll use the Census Bureau API to get tract boundaries and key social vulnerability indicators. These demographics will become our socioeconomic risk features:

```
def download_census_tracts(state_fips, county_fips):
    """
    Downloads census tract boundaries + ACS demographic variables
    for the target county. Returns a GeoDataFrame.
    """
    print('📥 Downloading census tract boundaries & demographics...')
```

```
    # ── Step 1: Get tract geometries from Census TIGER API ────────────────
    geo_url = ('https://tigerweb.geo.census.gov/arcgis/rest/services/'
               'TIGERweb/Tracts_Blocks/MapServer/8/query')
    geo_params = {
        'where': f"STATE='{state_fips}' AND COUNTY='{county_fips}'",
        'outFields': 'GEOID,NAME',
        'f': 'geojson',
        'returnGeometry': 'true',
        'outSR': '4326',
    }
    resp = requests.get(geo_url, params=geo_params, timeout=120)
    tracts = gpd.read_file(io.BytesIO(resp.content))
    tracts = tracts.to_crs(PROJ_CRS)

    # ── Step 2: Get ACS 5-year demographic data ──────────────────────────
    # Variables: B01003_001E = total pop, B19013_001E = median income,
    #            B17001_002E = below poverty, B25002_003E = vacant units
    acs_url = 'https://api.census.gov/data/2021/acs/acs5'
    variables = 'B01003_001E,B19013_001E,B17001_002E,B25002_003E'
    acs_params = {
        'get': variables,
        'for': f'tract:*',
        'in': f'state:{state_fips} county:{county_fips}',
    }
    acs_resp = requests.get(acs_url, params=acs_params)
    acs_data = acs_resp.json()
    acs_df = pd.DataFrame(acs_data[1:], columns=acs_data[0])
    acs_df['GEOID'] = acs_df['state'] + acs_df['county'] + acs_df['tract']

    # Rename columns to meaningful names
    acs_df = acs_df.rename(columns={
        'B01003_001E': 'total_pop',
        'B19013_001E': 'median_income',
        'B17001_002E': 'poverty_count',
        'B25002_003E': 'vacant_units',
    })
    # Convert to numeric (census API returns strings)
    for col in ['total_pop','median_income','poverty_count','vacant_units']:
        acs_df[col] = pd.to_numeric(acs_df[col], errors='coerce')
    acs_df['poverty_rate'] = acs_df['poverty_count'] / acs_df['total_pop']

    # ── Step 3: Merge geometry + demographics ────────────────────────────
    tracts = tracts.merge(acs_df[['GEOID','total_pop','median_income',
                                  'poverty_rate','vacant_units']],
                          on='GEOID', how='left')
    print(f'   ✅ Loaded {len(tracts)} census tracts')
    print(f'   Columns: {list(tracts.columns)}')
    return tracts

tracts = download_census_tracts(STATE_FIPS, COUNTY_FIPS)
```

✅ **Expected Output: Census Download**

```
📥 Downloading census tract boundaries & demographics...
   ✅ Loaded 786 census tracts
   Columns: ['GEOID', 'NAME', 'geometry', 'total_pop',
             'median_income', 'poverty_rate', 'vacant_units']
```

1.4 — Download Water Bodies via OpenStreetMap

We'll use OSMnx to download rivers, lakes, and bayous in Harris County. These are key hazard sources — proximity to water is one of the strongest flood risk predictors:

```
import osmnx as ox

def download_water_bodies(county_name):
    """
    Downloads rivers, streams, and lakes from OpenStreetMap.
    Returns a GeoDataFrame of water body geometries.
    """
    print('📥 Downloading water bodies from OpenStreetMap...')

    # OSMnx 'features' method fetches OpenStreetMap tagged features
    # 'waterway' captures rivers and streams (line geometries)
    waterways = ox.features_from_place(
        county_name,
        tags={'waterway': ['river', 'stream', 'canal', 'drain']}
    )
    waterways = waterways[['geometry']].reset_index(drop=True)
    waterways = waterways.to_crs(PROJ_CRS)

    print(f'   ✅ Downloaded {len(waterways):,} waterway features')
    return waterways

# Note: OSMnx uses place name as a string for geocoding
waterways = download_water_bodies('Harris County, Texas, USA')
```

✅ Expected Output: OSM Download

```
📥 Downloading water bodies from OpenStreetMap...
   ✅ Downloaded 3,241 waterway features
```

💡 OSMnx takes a few minutes

OpenStreetMap data downloads can take 2–5 minutes for a county-level area. This is normal — it's fetching live data from the OSM servers. If it fails, check your internet connection and try again.

1.5 — Quick Sanity Check: Visualize the Raw Data

Before doing any analysis, always plot your data to confirm it loaded correctly and looks geographically sensible:

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Harris County — Raw Data Check', fontsize=16, fontweight='bold')

# Plot 1: Census tracts colored by poverty rate
tracts.plot(ax=axes[0], column='poverty_rate', cmap='YlOrRd',
            legend=True, legend_kwds={'label': 'Poverty Rate'})
axes[0].set_title('Census Tracts — Poverty Rate')
axes[0].axis('off')

# Plot 2: FEMA flood zones
zone_colors = {'AE':'#ef4444','AO':'#f97316','X':'#3b82f6','D':'#9ca3af'}
flood_zones['color'] = flood_zones['FLD_ZONE'].map(zone_colors).fillna('#e5e7eb')
```

```
flood_zones.plot(ax=axes[1], color=flood_zones['color'])
axes[1].set_title('FEMA Flood Zones')
axes[1].axis('off')

# Plot 3: Waterways
tracts.plot(ax=axes[2], color='#e5e7eb', edgecolor='white', linewidth=0.3)
waterways.plot(ax=axes[2], color='#3b82f6', linewidth=0.5, alpha=0.7)
axes[2].set_title('OpenStreetMap Waterways')
axes[2].axis('off')

plt.tight_layout()
plt.savefig('01_raw_data_check.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ Raw data looks correct — three layers, all covering Harris County')
```

✅ Expected Output: Visualization

```
✅ Raw data looks correct — three layers, all covering Harris County

(A 3-panel figure appears showing: left = census tracts colored by
poverty rate; center = red/orange/blue flood zone polygons; right =
blue waterway lines on a gray county background)
```

STEP 2

Exploratory Data Analysis

Understand the data before modeling

⏱ Estimated time: 20–30 minutes

Before building a model, a good data scientist always explores the data first. This step helps you spot missing values, understand distributions, and form hypotheses about which features might be most predictive.

2.1 — Check for Missing Values

```
print('=== MISSING VALUE REPORT ===')
print()

# Check census tracts
print('--- Census Tracts ---')
missing =
tracts[['total_pop','median_income','poverty_rate','vacant_units']].isnull().sum()
print(missing)
print(f'Total tracts: {len(tracts)}')
print()

# Check for tracts with very low population (hard to analyze)
low_pop = tracts[tracts['total_pop'] < 50]
print(f'Tracts with population < 50 (will be excluded): {len(low_pop)}')

# Drop missing and low-population tracts
tracts_clean = tracts.dropna(subset=['total_pop','median_income','poverty_rate'])
tracts_clean = tracts_clean[tracts_clean['total_pop'] >= 50].copy()
print(f'\nClean tracts remaining: {len(tracts_clean)}')
```

✅ Expected Output: Missing Value Report

```
=== MISSING VALUE REPORT ===

--- Census Tracts ---
total_pop        0
median_income    8
poverty_rate     3
vacant_units     2
dtype: int64

Total tracts: 786
Tracts with population < 50 (will be excluded): 12

Clean tracts remaining: 769
```

ℹ Why exclude low-population tracts?

Census tracts with fewer than 50 people are often special areas like airports, industrial zones, or parks. Their demographic statistics are unreliable and they can distort our spatial analysis.

2.2 — Understand the Distribution of Key Variables

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Distribution of Key Variables — Harris County Census Tracts',
             fontsize=14, fontweight='bold')

plot_configs = [
    ('total_pop',     'Total Population per Tract',  'Population', '#3b82f6'),
    ('median_income', 'Median Household Income ($)', 'Income ($)',  '#10b981'),
    ('poverty_rate',  'Poverty Rate',                'Rate',        '#ef4444'),
    ('vacant_units',  'Vacant Housing Units',        'Units',       '#f59e0b'),
]

for ax, (col, title, xlabel, color) in zip(axes.flatten(), plot_configs):
    tracts_clean[col].hist(ax=ax, bins=30, color=color, alpha=0.8, edgecolor='white')
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Number of Tracts')
    # Add a vertical line at the mean
    ax.axvline(tracts_clean[col].mean(), color='black', linestyle='--',
               linewidth=1.5, label=f'Mean: {tracts_clean[col].mean():.1f}')
    ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig('02_distributions.png', dpi=150, bbox_inches='tight')
plt.show()

print('--- Summary Statistics ---')
print(tracts_clean[['total_pop','median_income','poverty_rate']].describe().round(2))
```

✅ Expected Output: Summary Statistics

```
--- Summary Statistics ---
          total_pop  median_income  poverty_rate
count    769.000000     769.000000    769.000000
mean    3721.450000   67342.000000      0.127000
std     2341.200000   41820.000000      0.117000
min       50.000000    2499.000000      0.000000
25%     2102.000000   37812.000000      0.038000
50%     3398.000000   58104.000000      0.091000
75%     4932.000000   89214.000000      0.183000
max    17823.000000  250001.000000      0.743000
```

STEP 3

Engineer Spatial Features

Create flood risk predictors using buffer analysis and spatial joins

⏱ Estimated time: 45–60 minutes

This is the core GIS step — transforming raw geographic layers into quantitative features that a machine learning model can use. We will create four spatial features for each census tract.

> **Concept: What is a Spatial Feature?**
> A spatial feature is a numeric value derived from geographic relationships. For example, 'area of the tract inside a FEMA AE flood zone' is a spatial feature — it tells the ML model how much of each tract overlaps the officially designated high-risk zone. Other examples: distance to the nearest river, count of waterways within 500m, average elevation. These are features the model will use to predict risk.

Feature 3.1 — FEMA Flood Zone Coverage (Spatial Intersection)

Calculate what percentage of each census tract falls within a FEMA high-risk flood zone (AE or AO). A tract that is 80% within a flood zone is clearly at higher risk than one at 5%.

```
def calculate_flood_zone_coverage(tracts, flood_zones):
    """
    For each census tract, calculates the fraction of its area
    that overlaps with high-risk FEMA flood zones (AE, AO, VE).
    """
    print('🔗 Calculating FEMA flood zone coverage...')

    # Filter to high-risk zones only
    high_risk =
flood_zones[flood_zones['FLD_ZONE'].isin(['AE','AO','VE','A'])].copy()

    # Dissolve all high-risk zones into one single polygon
    # (so we can do one clean intersection)
    high_risk_dissolved = high_risk.dissolve()

    # Calculate each tract's total area (in square meters)
    tracts = tracts.copy()
    tracts['tract_area_m2'] = tracts.geometry.area

    # Compute the intersection of each tract with the high-risk zone polygon
    # This gives us the portion of the tract that is in a flood zone
    intersection = gpd.overlay(tracts[['GEOID','geometry','tract_area_m2']],
                               high_risk_dissolved[['geometry']],
                               how='intersection')

    # Calculate the flooded area for each tract
    intersection['flood_area_m2'] = intersection.geometry.area
    flood_coverage =
intersection.groupby('GEOID')['flood_area_m2'].sum().reset_index()

    # Merge back and calculate the ratio (0 = no flood zone, 1 = entirely in zone)
    tracts = tracts.merge(flood_coverage, on='GEOID', how='left')
```

```
    tracts['flood_area_m2'] = tracts['flood_area_m2'].fillna(0)
    tracts['fema_flood_pct'] = tracts['flood_area_m2'] / tracts['tract_area_m2']
    tracts['fema_flood_pct'] = tracts['fema_flood_pct'].clip(0, 1)  # Cap at 100%

    print(f'   ✅ Done. Mean coverage: {tracts["fema_flood_pct"].mean():.1%}')
    print(f'   Tracts >50% in flood zone: {(tracts["fema_flood_pct"]>0.5).sum()}')
    return tracts

tracts_clean = calculate_flood_zone_coverage(tracts_clean, flood_zones)
```

✅ Expected Output: Flood Zone Coverage

```
🔗 Calculating FEMA flood zone coverage...
   ✅ Done. Mean coverage: 23.4%
   Tracts >50% in flood zone: 187
```

Feature 3.2 — Distance to Nearest Waterway (Buffer / Proximity)

📚 Concept: What is a Buffer?

A buffer is a zone of a given distance around a geographic feature. For example, a 500-meter buffer around a river is a polygon that covers all land within 500 meters of that river. We use the reverse concept here: we calculate how far each tract's centroid is from the nearest waterway. Closer = more risky.

```
def calculate_waterway_proximity(tracts, waterways):
    """
    For each census tract centroid, calculates the distance (in meters)
    to the nearest OSM waterway feature.
    """
    print('🔗 Calculating waterway proximity...')

    # Get the centroid of each tract (the geographic center point)
    tracts = tracts.copy()
    tract_centroids = tracts.geometry.centroid

    # Combine all waterway geometries into one union for efficient distance calc
    waterways_union = waterways.geometry.unary_union

    # Calculate distance from each tract centroid to the nearest waterway
    # .distance() returns distance in the CRS units — for EPSG:32615, that's
meters
    tracts['dist_to_water_m'] = tract_centroids.distance(waterways_union)

    # Log-transform the distance: a tract 100m away vs 200m away matters more
    # than one 5,000m away vs 5,100m away. Log captures this diminishing effect.
    tracts['log_dist_to_water'] = np.log1p(tracts['dist_to_water_m'])

    print(f'   ✅ Done.')
    print(f'   Min distance: {tracts["dist_to_water_m"].min():.0f}m')
    print(f'   Max distance: {tracts["dist_to_water_m"].max():.0f}m')
    print(f'   Mean distance: {tracts["dist_to_water_m"].mean():.0f}m')
    return tracts

tracts_clean = calculate_waterway_proximity(tracts_clean, waterways)
```

✅ **Expected Output: Waterway Proximity**

```
🔗 Calculating waterway proximity...
   ✅ Done.
   Min distance:  0m
   Max distance:  8,432m
   Mean distance: 1,214m
```

Feature 3.3 — Count Waterways Within 1km (Spatial Join with Buffer)

This feature captures waterway density around each tract, not just proximity to the nearest one. A tract surrounded by multiple bayous is more vulnerable than one next to a single small stream:

```
def count_waterways_in_buffer(tracts, waterways, buffer_m=1000):
    """
    Creates a buffer around each tract centroid and counts how
    many waterway features intersect that buffer zone.
    buffer_m: buffer radius in meters (default 1km = 1000m)
    """
    print(f'🔗 Counting waterways within {buffer_m}m buffer...')

    tracts = tracts.copy()

    # Create a 1km buffer circle around each tract centroid
    tract_buffers = tracts.copy()
    tract_buffers['geometry'] = tracts.geometry.centroid.buffer(buffer_m)

    # Spatial join: find which waterways fall inside each buffer
    # how='left' means: keep all tracts even if no waterways found
    # predicate='intersects' means: waterway touches or overlaps the buffer
    joined = gpd.sjoin(
        tract_buffers[['GEOID','geometry']],
        waterways.reset_index()[['geometry']],
        how='left',
        predicate='intersects'
    )

    # Count how many waterway features each tract intersected
    waterway_counts = joined.groupby('GEOID').size().reset_index(
        name='waterways_within_1km'
    )

    tracts = tracts.merge(waterway_counts, on='GEOID', how='left')
    tracts['waterways_within_1km'] = tracts['waterways_within_1km'].fillna(0)

    print(f'   ✅ Done. Mean waterway count:
{tracts["waterways_within_1km"].mean():.1f}')
    return tracts

tracts_clean = count_waterways_in_buffer(tracts_clean, waterways, buffer_m=1000)
```

✅ **Expected Output: Waterway Count**

```
🔗 Counting waterways within 1000m buffer...
   ✅ Done. Mean waterway count: 18.3
```

Feature 3.4 — Create the Target Variable (Composite Vulnerability Score)

Since we don't have historical flood insurance claim data for this tutorial, we will construct a rule-based composite vulnerability score as our target variable. This is a common technique when labeled data doesn't exist — you encode domain knowledge into a scoring formula. A professional project would replace this with real claims data.

```
def create_vulnerability_score(tracts):
    """
    Creates a composite flood vulnerability score (0-100) for each
    census tract using a weighted combination of spatial + socioeconomic
    risk factors.

    Score formula (domain-knowledge weights):
      40% — FEMA flood zone coverage (physical hazard exposure)
      25% — Proximity to water (normalized, inverted: closer = higher risk)
      15% — Poverty rate (social vulnerability: less ability to recover)
      20% — Waterway density within 1km (compound exposure risk)
    """
    tracts = tracts.copy()

    # Normalize each feature to 0-1 range using Min-Max scaling
    def minmax(series):
        return (series - series.min()) / (series.max() - series.min() + 1e-9)

    # Physical hazard: FEMA coverage (higher = worse, already 0-1)
    fema_score = tracts['fema_flood_pct']

    # Proximity: INVERT so closer to water = higher score
    max_dist = tracts['dist_to_water_m'].max()
    proximity_score = 1 - minmax(tracts['dist_to_water_m'])

    # Social vulnerability: poverty rate (normalized)
    poverty_score = minmax(tracts['poverty_rate'])

    # Waterway density (normalized)
    density_score = minmax(tracts['waterways_within_1km'])

    # Composite weighted score (0-100)
    tracts['vulnerability_score'] = (
        0.40 * fema_score +
        0.25 * proximity_score +
        0.15 * poverty_score +
        0.20 * density_score
    ) * 100

    # Assign risk tiers for labeling
    tracts['risk_tier'] = pd.cut(
        tracts['vulnerability_score'],
        bins=[0, 20, 40, 60, 80, 100],
        labels=['Very Low','Low','Medium','High','Very High']
    )

    print('✅ Vulnerability scores created')
    print(tracts['risk_tier'].value_counts())
    return tracts

tracts_clean = create_vulnerability_score(tracts_clean)
```

✅ **Expected Output: Vulnerability Score Distribution**

```
✅ Vulnerability scores created
risk_tier
Low          241
Medium       198
Very Low     162
High         127
Very High     41
dtype: int64
```

This is where domain expertise matters

The weights (40%, 25%, 15%, 20%) represent assumptions about how important each factor is. A real project would involve consulting with emergency managers or hydrologists to calibrate these weights, or would use real historical flood claim data as the ground truth label.

STEP 4

Spatial Autocorrelation Analysis

Test whether flood risk clusters geographically using Moran's I

⏱ Estimated time: 30–40 minutes

Concept: What is Spatial Autocorrelation?

Spatial autocorrelation asks: 'Do nearby places tend to have similar values?' If high-vulnerability tracts cluster together (surrounded by other high-vulnerability tracts), we say there is positive spatial autocorrelation. If they are randomly scattered, there is none. Moran's I is the standard statistic for measuring this: it ranges from -1 (perfect dispersion) through 0 (random) to +1 (perfect clustering). For flood risk, we expect high positive autocorrelation — flooded areas are not random, they follow waterways and low-lying terrain.

4.1 — Compute Global Moran's I

Global Moran's I gives one number describing the overall spatial autocorrelation pattern for the entire study area:

```
import esda
from libpysal.weights import Queen
from esda.moran import Moran, Moran_Local

# ── Step 1: Build a spatial weights matrix ──────────────────────────────
# The weights matrix defines which tracts are 'neighbors' of each other.
# Queen contiguity: two tracts are neighbors if they share any boundary point
# (includes corner-touching, unlike Rook which requires shared edges).

print('Building spatial weights matrix (Queen contiguity)...')
w = Queen.from_dataframe(tracts_clean, use_index=False)
w.transform = 'R'  # Row-standardize: each row sums to 1
print(f'✅ Weights matrix built: {w.n} tracts, mean neighbors:
{w.mean_neighbors:.1f}')
print()

# ── Step 2: Compute Global Moran's I ────────────────────────────────────
y = tracts_clean['vulnerability_score'].values
moran_global = Moran(y, w)

print('=== GLOBAL MORAN\'S I RESULT ===')
print(f'Moran\'s I statistic:  {moran_global.I:.4f}')
print(f'Expected value (H0): {moran_global.EI:.4f}  (random = 0)')
print(f'p-value:             {moran_global.p_sim:.4f}')
print()
if moran_global.p_sim < 0.05:
    print('✅ Statistically significant clustering detected (p < 0.05)!')
    print(f'   Interpretation: High-vulnerability tracts cluster together.')
    print(f'   This confirms spatial autocorrelation — geography matters.')
else:
    print('ℹ No statistically significant clustering detected.')
```

✅ Expected Output: Global Moran's I

```
Building spatial weights matrix (Queen contiguity)...
```

```
✅ Weights matrix built: 769 tracts, mean neighbors: 5.8

=== GLOBAL MORAN'S I RESULT ===
Moran's I statistic:  0.4821
Expected value (H0): -0.0013  (random = 0)
p-value:             0.0010

✅ Statistically significant clustering detected (p < 0.05)!
   Interpretation: High-vulnerability tracts cluster together.
   This confirms spatial autocorrelation — geography matters.
```

> **ℹ What does I = 0.48 mean?**
> A Moran's I of 0.48 (out of a maximum of 1.0) indicates moderate-to-strong spatial clustering. This means flood-vulnerable tracts are NOT randomly distributed — they form distinct geographic clusters, which is exactly what we'd expect: flood-prone areas follow the bayou network through the county.

4.2 — Local Moran's I (LISA Map) — Finding the Hotspots

While global Moran's I gives us one overall number, Local Moran's I (LISA — Local Indicators of Spatial Association) identifies WHERE the significant clusters are. This produces one of the most important maps in your portfolio:

```
# ── Compute Local Moran's I ──────────────────────────────────────────────
moran_local = Moran_Local(y, w, permutations=999)

# Add LISA results to our GeoDataFrame
tracts_clean = tracts_clean.copy()
tracts_clean['lisa_I']   = moran_local.Is       # Local I statistic
tracts_clean['lisa_p']   = moran_local.p_sim    # Simulated p-value
tracts_clean['lisa_q']   = moran_local.q        # Quadrant classification

# Classify into standard LISA quadrant labels
# Quadrant codes: 1=HH, 2=LH, 3=LL, 4=HL
# HH = High vulnerability surrounded by High — true hotspot
# LL = Low vulnerability surrounded by Low  — true coldspot
# HL = High surrounded by Low — spatial outlier
# LH = Low surrounded by High — spatial outlier
SIGNIFICANCE_THRESHOLD = 0.05  # Only show statistically significant results

quadrant_labels = {1: 'HH (Hotspot)', 2: 'LH (Outlier)',
                   3: 'LL (Coldspot)', 4: 'HL (Outlier)'}
quadrant_colors = {
    'HH (Hotspot)': '#dc2626',   # Red — high risk cluster
    'LH (Outlier)': '#f97316',   # Orange — low surrounded by high
    'LL (Coldspot)': '#3b82f6',  # Blue — low risk cluster
    'HL (Outlier)': '#93c5fd',   # Light blue — high surrounded by low
    'Not Significant': '#e5e7eb' # Gray — not statistically significant
}

tracts_clean['lisa_label'] = np.where(
    tracts_clean['lisa_p'] < SIGNIFICANCE_THRESHOLD,
    tracts_clean['lisa_q'].map(quadrant_labels),
    'Not Significant'
)

# ── Plot the LISA Map ────────────────────────────────────────────────────
```

```
fig, ax = plt.subplots(1, 1, figsize=(12, 10))

for label, color in quadrant_colors.items():
    subset = tracts_clean[tracts_clean['lisa_label'] == label]
    subset.plot(ax=ax, color=color, edgecolor='white', linewidth=0.2,
                label=f'{label} (n={len(subset)})')

ax.set_title('Local Moran\'s I (LISA) — Flood Vulnerability Clusters\n'
             'Harris County, Texas', fontsize=14, fontweight='bold')
ax.legend(loc='lower right', fontsize=10, title='Cluster Type')
ax.axis('off')

plt.tight_layout()
plt.savefig('03_lisa_map.png', dpi=150, bbox_inches='tight')
plt.show()

print('✅ LISA map saved as: 03_lisa_map.png')
print('\nLISA cluster summary:')
print(tracts_clean['lisa_label'].value_counts())
```

✅ Expected Output: LISA Summary

```
✅ LISA map saved as: 03_lisa_map.png

LISA cluster summary:
lisa_label
Not Significant    441
LL (Coldspot)      152
HH (Hotspot)       112
HL (Outlier)        36
LH (Outlier)        28
dtype: int64
```

💡 This map IS your portfolio showpiece

The LISA map is one of the most visually striking and analytically meaningful outputs in geospatial data science. The red HH clusters show you exactly where flood vulnerability concentrates — typically along the Buffalo Bayou and Brays Bayou corridors in Houston. Include this map in your GitHub README and LinkedIn post.

STEP 5

Train the ML Model

Use Random Forest to predict vulnerability scores from spatial features

⏱ Estimated time: 30–40 minutes

Concept: Why Random Forest?

Random Forest is an ensemble of many decision trees. It works well for spatial data because it handles non-linear relationships (e.g., flood risk doesn't increase linearly with proximity to water), it's robust to outliers and missing values, and it provides feature importance scores that tell you which predictors mattered most. It's also interpretable enough to explain to a non-technical audience.

5.1 — Prepare the Feature Matrix

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.preprocessing import StandardScaler

# ── Define features (X) and target (y) ──────────────────────────────────
FEATURE_COLS = [
    'fema_flood_pct',       # Physical hazard: FEMA flood zone coverage
    'dist_to_water_m',      # Distance to nearest waterway (meters)
    'log_dist_to_water',    # Log-transformed distance (captures diminishing
return)
    'waterways_within_1km', # Waterway density within 1km
    'median_income',        # Socioeconomic: household income
    'poverty_rate',         # Socioeconomic: poverty rate
    'total_pop',            # Population (larger pop = more people at risk)
    'vacant_units',         # Housing: vacant units (proxy for disinvestment)
]
TARGET_COL = 'vulnerability_score'

# Drop rows with any NaN in our feature or target columns
model_df = tracts_clean[FEATURE_COLS + [TARGET_COL]].dropna()
print(f'Rows available for modeling: {len(model_df)}')

X = model_df[FEATURE_COLS].values
y = model_df[TARGET_COL].values

# ── Train/Test Split ────────────────────────────────────────────────────
# IMPORTANT NOTE for GIS analysts: for a publishable analysis, you should
# use spatial cross-validation (e.g., spatial k-fold) instead of random
# split, to avoid data leakage between spatially adjacent tracts.
# For this beginner guide, we use a simple 80/20 random split.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
print(f'Training set: {len(X_train)} tracts')
print(f'Test set:     {len(X_test)} tracts')
```

✅ Expected Output: Data Split

```
Rows available for modeling: 751
Training set: 600 tracts
```

```
Test set:     151 tracts
```

5.2 — Train the Random Forest Model

```
# ── Train the model ──────────────────────────────────────────────────────
print('🤖 Training Random Forest model...')

rf_model = RandomForestRegressor(
    n_estimators=200,   # Number of trees (more = more stable, slower to train)
    max_depth=8,        # Maximum depth of each tree (prevents overfitting)
    min_samples_leaf=5, # Each leaf must have at least 5 samples
    random_state=42,    # For reproducibility — always set this!
    n_jobs=-1           # Use all CPU cores for faster training
)

rf_model.fit(X_train, y_train)
print('✅ Model trained!')

# ── Evaluate on the test set ─────────────────────────────────────────────
y_pred = rf_model.predict(X_test)

r2  = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print()
print('=== MODEL EVALUATION (Test Set) ===')
print(f'R² Score:                  {r2:.4f}')
print(f'  → Explains {r2*100:.1f}% of variance in vulnerability scores')
print(f'Mean Absolute Error (MAE): {mae:.2f} points')
print(f'  → On average, predictions are off by {mae:.1f}/100 score points')

# ── Visualize predictions vs actuals ─────────────────────────────────────
fig, ax = plt.subplots(figsize=(7, 6))
ax.scatter(y_test, y_pred, alpha=0.5, s=20, color='#3b82f6')
ax.plot([0, 100], [0, 100], 'r--', linewidth=1.5, label='Perfect prediction')
ax.set_xlabel('Actual Vulnerability Score', fontsize=12)
ax.set_ylabel('Predicted Vulnerability Score', fontsize=12)
ax.set_title(f'Random Forest Predictions vs Actual\nR² = {r2:.3f} | MAE =
{mae:.1f}', fontsize=12)
ax.legend()
ax.set_xlim(0, 100); ax.set_ylim(0, 100)
plt.tight_layout()
plt.savefig('04_predictions_vs_actual.png', dpi=150, bbox_inches='tight')
plt.show()
```

✅ Expected Output: Model Performance

```
🤖 Training Random Forest model...
✅ Model trained!

=== MODEL EVALUATION (Test Set) ===
R² Score:                  0.9124
  → Explains 91.2% of variance in vulnerability scores
Mean Absolute Error (MAE): 3.47 points
  → On average, predictions are off by 3.5/100 score points
```

ℹ Why is R² so high?

> $R^2 = 0.91$ looks impressive, but remember: the target variable (vulnerability score) was constructed FROM these same features using the formula in Step 3.4. A real project would have a harder, independent target (like actual flood insurance payouts). Even so, demonstrating the full ML pipeline — including proper train/test splitting and evaluation — is the key portfolio skill here.

5.3 — Feature Importance: What Drives Flood Vulnerability?

One of the most useful outputs of Random Forest is feature importance — which variables contributed most to the model's predictions:

```
# ── Feature Importance Plot ──────────────────────────────────────────────
importances = rf_model.feature_importances_
feat_df = pd.DataFrame({
    'Feature': FEATURE_COLS,
    'Importance': importances
}).sort_values('Importance', ascending=True)

fig, ax = plt.subplots(figsize=(9, 5))
colors = ['#3b82f6' if i < len(feat_df)-3 else '#ef4444'
          for i in range(len(feat_df))]
bars = ax.barh(feat_df['Feature'], feat_df['Importance'],
               color=colors, edgecolor='white')

# Add value labels on the bars
for bar, val in zip(bars, feat_df['Importance']):
    ax.text(val + 0.002, bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=9)

ax.set_title('Random Forest — Feature Importance\nWhat predicts flood
vulnerability?',
             fontsize=13, fontweight='bold')
ax.set_xlabel('Importance Score (higher = more influential)')
ax.axvline(x=1/len(FEATURE_COLS), color='gray', linestyle='--',
           alpha=0.5, label='Equal importance baseline')
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig('05_feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

print('\nTop 3 most important features:')
top3 = feat_df.nlargest(3, 'Importance')
for _, row in top3.iterrows():
    print(f'  {row["Feature"]:<25} {row["Importance"]:.3f}')
```

✅ Expected Output: Feature Importance

```
Top 3 most important features:
  fema_flood_pct            0.412
  dist_to_water_m           0.231
  waterways_within_1km      0.178
```

💡 Interpreting feature importance

FEMA flood zone coverage (0.41) being the top predictor makes intuitive sense — the official flood zone designation captures decades of hydrological engineering study. The fact that our

> OSM proximity measure (0.23) is the second-strongest suggests that simple distance-to-water is a powerful, easy-to-compute risk signal even without the official flood zone data.

STEP 6

Build the Interactive Web Map

Create a shareable Folium choropleth map

⏱ Estimated time: 30–40 minutes

The final deliverable is an interactive HTML map that anyone can open in a browser. Stakeholders can hover over tracts to see vulnerability scores, zoom in on hotspots, and toggle layers.

6.1 — Add ML Predictions Back to the GeoDataFrame

```
# Add model predictions to the full clean dataset
# (We trained on a subset, now predict for ALL tracts)
X_all = tracts_clean[FEATURE_COLS].fillna(0).values
tracts_clean['predicted_score'] = rf_model.predict(X_all)
tracts_clean['predicted_score'] = tracts_clean['predicted_score'].clip(0, 100)

# Also assign a predicted risk tier label
tracts_clean['predicted_tier'] = pd.cut(
    tracts_clean['predicted_score'],
    bins=[0, 20, 40, 60, 80, 100],
    labels=['Very Low','Low','Medium','High','Very High']
)

print('✅ Predictions added for all tracts')
print(tracts_clean['predicted_tier'].value_counts())
```

✅ Expected Output: Predictions for All Tracts

```
✅ Predictions added for all tracts
predicted_tier
Low          248
Medium       201
Very Low     156
High         121
Very High     23
dtype: int64
```

6.2 — Build the Folium Choropleth Map

```
# Convert to WGS84 (lat/lon) for web mapping — Folium needs EPSG:4326
tracts_web = tracts_clean.to_crs('EPSG:4326')

# ── Initialize the Folium map ──────────────────────────────────────────
# Center on Harris County, TX
m = folium.Map(
    location=[29.78, -95.37],
    zoom_start=10,
    tiles='CartoDB positron',   # Clean, minimal basemap
    attr='CartoDB'
)

# ── Add the vulnerability choropleth layer ─────────────────────────────
choropleth = folium.Choropleth(
    geo_data=tracts_web.__geo_interface__,
```

```
    name='Flood Vulnerability Score',
    data=tracts_web,
    columns=['GEOID', 'predicted_score'],
    key_on='feature.properties.GEOID',
    fill_color='YlOrRd',         # Yellow → Orange → Red color ramp
    fill_opacity=0.75,
    line_opacity=0.3,
    line_weight=0.5,
    legend_name='Flood Vulnerability Score (0-100)',
    nan_fill_color='lightgray',
    highlight=True,              # Highlight tract on hover
).add_to(m)

# ── Add tooltip: hover over a tract to see details ──────────────────────
tooltip_cols = ['GEOID', 'predicted_score', 'predicted_tier',
                'fema_flood_pct', 'poverty_rate', 'median_income',
                'total_pop', 'lisa_label']

# Round numerical columns for display
display_df = tracts_web.copy()
display_df['predicted_score'] = display_df['predicted_score'].round(1)
display_df['fema_flood_pct']  = (display_df['fema_flood_pct'] * 100).round(1)
display_df['poverty_rate']    = (display_df['poverty_rate'] * 100).round(1)
display_df['median_income']   = display_df['median_income'].apply(
    lambda x: f'${x:,.0f}' if pd.notna(x) else 'N/A'
)

folium.GeoJson(
    display_df.__geo_interface__,
    style_function=lambda x: {'fillOpacity': 0, 'weight': 0},
    tooltip=folium.GeoJsonTooltip(
        fields=['predicted_score', 'predicted_tier', 'fema_flood_pct',
                'poverty_rate', 'median_income', 'total_pop', 'lisa_label'],
        aliases=['🎯 Vulnerability Score:', '⚠️ Risk Tier:',
                 '🏛️ FEMA Flood Zone %:', '📉 Poverty Rate %:',
                 '💰 Median Income:', '👥 Population:',
                 '📍 Cluster Type:'],
        localize=True,
        sticky=False,
        style='font-family: Arial; font-size: 12px;'
    )
).add_to(m)

# ── Add LISA hotspot layer as a toggle ──────────────────────────────────
lisa_colors = {
    'HH (Hotspot)': '#dc2626',
    'LL (Coldspot)': '#3b82f6',
    'HL (Outlier)': '#93c5fd',
    'LH (Outlier)': '#f97316',
    'Not Significant': 'transparent'
}

folium.GeoJson(
    display_df.__geo_interface__,
    name='LISA Clusters (toggle)',
    style_function=lambda feature: {
        'fillColor': lisa_colors.get(feature['properties']['lisa_label'], 'gray'),
        'fillOpacity': 0.7 if feature['properties']['lisa_label'] != 'Not
Significant' else 0,
        'weight': 0.5,
        'color': '#374151',
    },
    show=False,   # Hidden by default — user can toggle on
).add_to(m)
```

```
# ── Add layer control ──────────────────────────────────────────────
folium.LayerControl(position='topright', collapsed=False).add_to(m)

# ── Save to HTML file ──────────────────────────────────────────────
output_path = 'flood_vulnerability_harris_county.html'
m.save(output_path)
print(f'✅ Interactive map saved: {output_path}')
print(f'   Open this file in any web browser to explore the map.')
print(f'   Upload it to GitHub Pages to share it publicly.')
```

✅ Expected Output: Map Saved

```
✅ Interactive map saved: flood_vulnerability_harris_county.html
   Open this file in any web browser to explore the map.
   Upload it to GitHub Pages to share it publicly.
```

STEP 7

Interpret & Document Your Results

Write up your findings and prepare your portfolio deliverable

⏱ Estimated time: 30 minutes

The analysis is complete — now the critical step that many beginners skip: documenting what you found and what it means. A GitHub repo with code and no write-up is half a portfolio piece. With a write-up, it becomes a full one.

7.1 — Key Findings to Document

Your write-up should address these questions using the numbers from your analysis:

Question to Answer	Where to Find the Answer
How many tracts are in the 'High' or 'Very High' risk tier?	Step 3.4 output — risk_tier value_counts()
Where do the HH hotspots concentrate geographically?	Step 4.2 LISA map — describe what neighborhoods/areas
What single factor most strongly predicts flood vulnerability?	Step 5.3 — top feature importance variable
Is there a correlation between poverty and flood risk?	Compare poverty_rate values in High vs Very Low risk tracts
What is the model's accuracy and what are its limitations?	Step 5.2 R^2 / MAE + discussion of synthetic target variable

7.2 — GitHub README Template

Use this structure for your GitHub repository README (write in Markdown format):

```
# 🌊 Urban Flood Risk & Vulnerability Mapping — Harris County, TX

## Overview
This project builds an end-to-end spatial machine learning pipeline
that predicts flood vulnerability at the census tract level using
FEMA flood zone data, OpenStreetMap waterways, and Census demographics.

**Key finding:** [Your Moran's I value] indicates significant spatial
clustering of flood risk. [N] tracts (representing [X] people) fall
in the High or Very High risk tier, concentrated primarily along the
[bayou/waterway names from your LISA map].

## Outputs
- 📊 `flood_vulnerability_harris_county.html` — Interactive web map
- 🗺️ `03_lisa_map.png` — LISA hotspot cluster map
- 📈 `05_feature_importance.png` — ML feature importance chart
- 📓 `flood_risk_analysis.ipynb` — Full reproducible notebook
```

```
## Methodology
1. Data collection: FEMA NFHL API, Census TIGERweb API, OSMnx
2. Feature engineering: FEMA flood zone % coverage, distance to
   nearest waterway, waterway density within 1km
3. Spatial analysis: Global & Local Moran's I (Queen contiguity weights)
4. ML modeling: Random Forest Regressor (R² = [your value])
5. Visualization: Folium interactive choropleth + LISA map

## Data Sources (All Free & Open)
- FEMA National Flood Hazard Layer: hazards.fema.gov
- US Census Bureau ACS 5-Year Estimates: api.census.gov
- OpenStreetMap via OSMnx

## Skills Demonstrated
GeoPandas · Spatial joins · Buffer analysis · Moran's I · LISA ·
Random Forest · Feature importance · Folium interactive maps
```

7.3 — Discussing Limitations (Shows Analytical Maturity)

Always include a limitations section. Employers and reviewers look for this as a sign of intellectual honesty:

> **Limitations to include in your write-up**
>
> 1. Synthetic target variable: The vulnerability score was constructed from the same features used to predict it. A production model would use historical flood insurance claims or FEMA damage assessments as ground truth.
> 2. Spatial autocorrelation in residuals: We used a simple random train/test split, which can cause data leakage between adjacent tracts. A more rigorous analysis would use spatial k-fold cross-validation.
> 3. Static snapshot: This model reflects one point in time. Real flood risk changes with climate, land use change, and infrastructure improvements.
> 4. MAUP: Results would differ if we aggregated to block groups or ZIP codes instead of census tracts — always state your analysis unit.

Next Steps & Extensions

Once you've completed the core project, here are extensions that will deepen the analysis and further differentiate your portfolio:

Extension A — Add Elevation Data

Download USGS National Elevation Dataset (NED) rasters using the `elevation` or `rioxarray` library. Extract the mean elevation and terrain slope for each census tract. Low elevation + high slope toward waterways is a compound risk factor not captured by the current model.

```
# Install elevation library
!pip install elevation rioxarray
import elevation

# Download SRTM DEM for Harris County bounding box
# (29.45N, 94.85W) to (30.15N, 95.95W)
elevation.clip(bounds=(-95.95, 29.45, -94.85, 30.15),
               output='harris_dem.tif')

# Then use rioxarray to extract mean elevation per census tract
# (See rioxarray documentation for zonal statistics)
```

Extension B — Spatial Cross-Validation

Replace the random 80/20 split with proper spatial k-fold cross-validation to get unbiased model performance estimates. This shows graduate-level methodological rigor:

```
# Install spatial cross-validation library
!pip install sklearn-contrib-py-earth  # or use manual spatial folds

from libpysal.weights import KNN

# Manual spatial k-fold: partition tracts into k=5 geographic regions
# by k-means clustering on the tract centroids
from sklearn.cluster import KMeans
coords = np.column_stack([
    tracts_clean.geometry.centroid.x,
    tracts_clean.geometry.centroid.y
])
spatial_folds = KMeans(n_clusters=5, random_state=42).fit_predict(coords)
tracts_clean['spatial_fold'] = spatial_folds

# Now iterate: each fold is spatially contiguous, reducing data leakage
# (implement cross-validation loop here using spatial_fold column)
```

Extension C — Deploy as an ArcGIS StoryMap

Export your LISA map and interactive choropleth to ArcGIS Online (free with an ArcGIS Developer account) and build a StoryMap that walks viewers through your methodology and findings. StoryMaps are far more visible to GIS employers than a GitHub notebook.

> **Sharing the interactive map for free**
> Save the Folium map as an HTML file and host it for free using GitHub Pages. In your GitHub repo: Settings → Pages → Deploy from main branch. Your map becomes publicly accessible at: https://[yourusername].github.io/[repo-name]/flood_vulnerability_harris_county.html

Troubleshooting Common Errors

CRSError: Input shapes do not match CRS

You are trying to do a spatial operation (overlay, sjoin, buffer) between two GeoDataFrames with different coordinate systems. Fix: call .to_crs(PROJ_CRS) on all GeoDataFrames before any spatial operation.

requests.exceptions.ConnectionError / Timeout

The FEMA or Census API is unavailable or your internet is slow. Fix: add timeout=120 to your requests.get() call, or retry. If consistently failing, download the data manually from the FEMA website and load locally with gpd.read_file().

ValueError: Input contains NaN — from sklearn

Your feature matrix X has missing values. Fix: add .fillna(0) or .dropna() before creating the X array. Check which column has NaNs: model_df[FEATURE_COLS].isnull().sum()

gpd.overlay() takes very long / crashes

Overlaying detailed polygon layers is computationally expensive. Fix: simplify geometries first: flood_zones['geometry'] = flood_zones.geometry.simplify(tolerance=50). This reduces vertex count without meaningfully changing results.

Folium map shows as blank / grey

Usually a CRS issue. Fix: ensure you called .to_crs('EPSG:4326') before building the Folium map. Folium always expects lat/lon coordinates, not projected meters.

Glossary of Key Terms

Buffer	A zone of specified distance around a geographic feature (point, line, or polygon). Used to measure 'within X meters of Y'.
Choropleth Map	A map where geographic areas are colored based on a variable value. Common in census mapping (e.g., income by county). We use this for our vulnerability score.
CRS (Coordinate Reference System)	The mathematical framework that defines how geographic coordinates (lat/lon or meters) relate to the Earth. EPSG:4326 = lat/lon degrees; EPSG:32615 = UTM Zone 15N in meters.
FEMA NFHL	Federal Emergency Management Agency National Flood Hazard Layer — the official US government dataset of flood zones, used by insurance companies and banks for flood risk determination.
GeoDataFrame	A Pandas DataFrame with an added 'geometry' column containing geographic shapes. GeoPandas' core data structure for spatial analysis in Python.
LISA (Local Indicators of Spatial Association)	A per-location version of Moran's I. Identifies specific geographic clusters of high-high, low-low values and spatial outliers.
MAUP (Modifiable Areal Unit Problem)	The phenomenon where analysis results change when you change the geographic unit (e.g., tract vs. block group vs. ZIP code). A key concept in spatial analysis.
Moran's I	A global statistic for spatial autocorrelation. Ranges from -1 (perfect dispersion) to +1 (perfect clustering). Near 0 = random spatial pattern.
Random Forest	An ensemble ML algorithm that trains many decision trees and averages their predictions. Good for spatial data due to robustness and built-in feature importance.
Spatial Autocorrelation	The tendency for nearby locations to have similar values. Tobler's First Law: 'Everything is related to everything else, but near things are more related than distant things.'

Spatial Join	Combining two GeoDataFrames based on their geographic relationship (e.g., 'within', 'intersects', 'contains') rather than a shared ID column like a regular join.

Project Completion Checklist

Before publishing, make sure you have all of these completed:

✓	DELIVERABLE CHECKLIST
☐	**[Code]** All code cells run top-to-bottom without errors in a clean kernel restart
☐	**[Data]** All 3 data sources (FEMA, Census, OSM) downloaded and loaded successfully
☐	**[Visualization]** 3-panel raw data check plot saved (01_raw_data_check.png)
☐	**[Visualization]** Variable distribution histograms saved (02_distributions.png)
☐	**[Spatial Stats]** Global Moran's I computed — value and p-value documented in notebook
☐	**[Spatial Stats]** LISA map saved with HH/LL/HL/LH cluster labels (03_lisa_map.png)
☐	**[ML Model]** Random Forest trained, R^2 and MAE recorded in notebook
☐	**[ML Model]** Predictions vs. actuals scatter plot saved (04_predictions_vs_actual.png)
☐	**[ML Model]** Feature importance plot saved (05_feature_importance.png)
☐	**[Final Output]** Interactive HTML map saved and opens correctly in a browser
☐	**[Portfolio]** GitHub repo created with README covering methodology and findings
☐	**[Portfolio]** Key findings paragraph written: what did you discover about Harris County?
☐	**[Portfolio]** Limitations section included in notebook or README

Well done for completing Project 01. This is a real geospatial data science pipeline.

The combination of spatial operations, statistical analysis, machine learning, and interactive visualization you practiced here is exactly the skillset that separates GIS analysts who can do spatial data science from those who can only operate GIS software. Keep building!